

```
# Uninstall the currently installed Transformers library
!pip uninstall -y transformers

# Install a specific version of Transformers (4.46.3)
# along with its required dependencies
!pip install -U transformers==4.46.3 \
    accelerate \
    sentencepiece \
    huggingface-hub==0.26.2
```

```
Found existing installation: transformers 4.46.3
Uninstalling transformers-4.46.3:
  Successfully uninstalled transformers-4.46.3
Collecting transformers==4.46.3
  Using cached transformers-4.46.3-py3-none-any.whl.metadata (44 kB)
Requirement already satisfied: accelerate in /usr/local/lib/python3.12/dist-packages (1.14.0)
Requirement already satisfied: sentencepiece in /usr/local/lib/python3.12/dist-packages (0.2.2)
Requirement already satisfied: huggingface-hub==0.26.2 in /usr/local/lib/python3.12/dist-packages (0.26.2)
Requirement already satisfied: filelock in /usr/local/lib/python3.12/dist-packages (from transformers==4.46.3) (
Requirement already satisfied: numpy>=1.17 in /usr/local/lib/python3.12/dist-packages (from transformers==4.46.3)
Requirement already satisfied: packaging>=20.0 in /usr/local/lib/python3.12/dist-packages (from transformers==4.
Requirement already satisfied: pyyaml>=5.1 in /usr/local/lib/python3.12/dist-packages (from transformers==4.46.3)
Requirement already satisfied: regex!=2019.12.17 in /usr/local/lib/python3.12/dist-packages (from transformers==
Requirement already satisfied: requests in /usr/local/lib/python3.12/dist-packages (from transformers==4.46.3) (
Requirement already satisfied: tokenizers<0.21,>=0.20 in /usr/local/lib/python3.12/dist-packages (from transform
Requirement already satisfied: safetensors>=0.4.1 in /usr/local/lib/python3.12/dist-packages (from transformers=
Requirement already satisfied: tqdm>=4.27 in /usr/local/lib/python3.12/dist-packages (from transformers==4.46.3)
Requirement already satisfied: fsspec>=2023.5.0 in /usr/local/lib/python3.12/dist-packages (from huggingface-hub
Requirement already satisfied: typing-extensions>=3.7.4.3 in /usr/local/lib/python3.12/dist-packages (from huggi
Requirement already satisfied: psutil in /usr/local/lib/python3.12/dist-packages (from accelerate) (5.9.5)
Requirement already satisfied: torch>=2.0.0 in /usr/local/lib/python3.12/dist-packages (from accelerate) (2.11.0)
Requirement already satisfied: setuptools<82 in /usr/local/lib/python3.12/dist-packages (from torch>=2.0.0->acce
Requirement already satisfied: sympy>=1.13.3 in /usr/local/lib/python3.12/dist-packages (from torch>=2.0.0->acce
Requirement already satisfied: networkx>=2.5.1 in /usr/local/lib/python3.12/dist-packages (from torch>=2.0.0->ac
Requirement already satisfied: Jinja2 in /usr/local/lib/python3.12/dist-packages (from torch>=2.0.0->accelerate)
Requirement already satisfied: charset-normalizer<4,>=2 in /usr/local/lib/python3.12/dist-packages (from request
Requirement already satisfied: idna<4,>=2.5 in /usr/local/lib/python3.12/dist-packages (from requests->transform
Requirement already satisfied: urllib3<3,>=1.21.1 in /usr/local/lib/python3.12/dist-packages (from requests->tra
Requirement already satisfied: certifi>=2017.4.17 in /usr/local/lib/python3.12/dist-packages (from requests->tra
Requirement already satisfied: mpmath<1.4,>=1.1.0 in /usr/local/lib/python3.12/dist-packages (from sympy>=1.13.3)
Requirement already satisfied: MarkupSafe>=2.0 in /usr/local/lib/python3.12/dist-packages (from Jinja2->torch>=2
Using cached transformers-4.46.3-py3-none-any.whl (10.0 MB)
Installing collected packages: transformers
Successfully installed transformers-4.46.3
```

```
# Load the load_dataset from datasets library
from datasets import load_dataset
# Load our rotten_tomatoes movie review dataset
data = load_dataset("rotten_tomatoes")
# display the data
data
```

```
/usr/local/lib/python3.12/dist-packages/huggingface_hub/utils/_auth.py:94: UserWarning:
The secret `HF_TOKEN` does not exist in your Colab secrets.
To authenticate with the Hugging Face Hub, create a token in your settings tab (https://huggingface.co/settings)
You will be able to reuse this secret in all of your notebooks.
Please note that authentication is recommended but still optional to access public models or datasets.
  warnings.warn(
DatasetDict({
  train: Dataset({
    features: ['text', 'label'],
    num_rows: 8530
  })
  validation: Dataset({
    features: ['text', 'label'],
    num_rows: 1066
  })
  test: Dataset({
    features: ['text', 'label'],
    num_rows: 1066
```

```
    })
  })
```

```
# Get the first and last examples from the training dataset
data["train"][0, -1]]
```

```
{'text': ['the rock is destined to be the 21st century\'s new " conan " and that he\'s going to make a splash
even greater than arnold schwarzenegger , jean-claud van damme or steven segal .'],
 'things really get weird , though not particularly scary : the movie is all portent and no content .'],
 'label': [1, 0]}
```

```
# Uninstall any existing versions of TensorFlow, Keras, and tf-keras
# This helps avoid version conflicts.
#TensorFlow is the engine that performs all the mathematical computations needed to train and run AI models.
!pip uninstall -y tf-keras keras tensorflow tensorflow-cpu
```

```
# Install TensorFlow version 2.20.0
# Install tf-keras, which provides compatibility with some Hugging Face models
!pip install tensorflow==2.20.0 tf-keras
```

```
Successfully uninstalled tf-keras 2.20.0
Found existing installation: keras 3.15.0
Uninstalling keras-3.15.0:
  Successfully uninstalled keras-3.15.0
Found existing installation: tensorflow 2.20.0
Uninstalling tensorflow-2.20.0:
  Successfully uninstalled tensorflow-2.20.0
WARNING: Skipping tensorflow-cpu as it is not installed.
Collecting tensorflow==2.20.0
  Using cached tensorflow-2.20.0-cp312-cp312-manylinux_2_17_x86_64.manylinux2014_x86_64.whl.metadata (4.5 kB)
Collecting tf-keras
  Using cached tf_keras-2.21.0-py3-none-any.whl.metadata (1.8 kB)
Requirement already satisfied: absl-py>=1.0.0 in /usr/local/lib/python3.12/dist-packages (from tensorflow==2.20.0)
Requirement already satisfied: astunparse>=1.6.0 in /usr/local/lib/python3.12/dist-packages (from tensorflow==2.20.0)
Requirement already satisfied: flatbuffers>=24.3.25 in /usr/local/lib/python3.12/dist-packages (from tensorflow==2.20.0)
Requirement already satisfied: gast!=0.5.0,!0.5.1,!0.5.2,>=0.2.1 in /usr/local/lib/python3.12/dist-packages (from tensorflow==2.20.0)
Requirement already satisfied: google_pasta>=0.1.1 in /usr/local/lib/python3.12/dist-packages (from tensorflow==2.20.0)
Requirement already satisfied: libclang>=13.0.0 in /usr/local/lib/python3.12/dist-packages (from tensorflow==2.20.0)
Requirement already satisfied: opt_einsum>=2.3.2 in /usr/local/lib/python3.12/dist-packages (from tensorflow==2.20.0)
Requirement already satisfied: packaging in /usr/local/lib/python3.12/dist-packages (from tensorflow==2.20.0)
Requirement already satisfied: protobuf>=5.28.0 in /usr/local/lib/python3.12/dist-packages (from tensorflow==2.20.0)
Requirement already satisfied: requests<3,>=2.21.0 in /usr/local/lib/python3.12/dist-packages (from tensorflow==2.20.0)
Requirement already satisfied: setuptools in /usr/local/lib/python3.12/dist-packages (from tensorflow==2.20.0)
Requirement already satisfied: six>=1.12.0 in /usr/local/lib/python3.12/dist-packages (from tensorflow==2.20.0)
Requirement already satisfied: termcolor>=1.1.0 in /usr/local/lib/python3.12/dist-packages (from tensorflow==2.20.0)
Requirement already satisfied: typing_extensions>=3.6.6 in /usr/local/lib/python3.12/dist-packages (from tensorflow==2.20.0)
Requirement already satisfied: wrapt>=1.11.0 in /usr/local/lib/python3.12/dist-packages (from tensorflow==2.20.0)
Requirement already satisfied: grpcio<2.0,>=1.24.3 in /usr/local/lib/python3.12/dist-packages (from tensorflow==2.20.0)
Requirement already satisfied: tensorboard~=2.20.0 in /usr/local/lib/python3.12/dist-packages (from tensorflow==2.20.0)
Collecting keras>=3.10.0 (from tensorflow==2.20.0)
  Using cached keras-3.15.0-py3-none-any.whl.metadata (6.3 kB)
Requirement already satisfied: numpy>=1.26.0 in /usr/local/lib/python3.12/dist-packages (from keras>=3.10.0->tensorflow==2.20.0)
Requirement already satisfied: h5py>=3.11.0 in /usr/local/lib/python3.12/dist-packages (from keras>=3.10.0->tensorflow==2.20.0)
Requirement already satisfied: ml_dtypes<1.0.0,>=0.5.1 in /usr/local/lib/python3.12/dist-packages (from keras>=3.10.0->tensorflow==2.20.0)
INFO: pip is looking at multiple versions of tf-keras to determine which version is compatible with other required packages
Collecting tf-keras
  Using cached tf_keras-2.20.1-py3-none-any.whl.metadata (1.8 kB)
Requirement already satisfied: wheel<1.0,>=0.23.0 in /usr/local/lib/python3.12/dist-packages (from astunparse>=1.6.0->tensorflow==2.20.0)
Requirement already satisfied: rich in /usr/local/lib/python3.12/dist-packages (from keras>=3.10.0->tensorflow==2.20.0)
Requirement already satisfied: namex in /usr/local/lib/python3.12/dist-packages (from keras>=3.10.0->tensorflow==2.20.0)
Requirement already satisfied: optree in /usr/local/lib/python3.12/dist-packages (from keras>=3.10.0->tensorflow==2.20.0)
Requirement already satisfied: charset_normalizer<4,>=2 in /usr/local/lib/python3.12/dist-packages (from requests<3,>=2.21.0->tensorflow==2.20.0)
Requirement already satisfied: idna<4,>=2.5 in /usr/local/lib/python3.12/dist-packages (from requests<3,>=2.21.0->tensorflow==2.20.0)
Requirement already satisfied: urllib3<3,>=1.21.1 in /usr/local/lib/python3.12/dist-packages (from requests<3,>=2.21.0->tensorflow==2.20.0)
Requirement already satisfied: certifi>=2017.4.17 in /usr/local/lib/python3.12/dist-packages (from requests<3,>=2.21.0->tensorflow==2.20.0)
Requirement already satisfied: markdown>=2.6.8 in /usr/local/lib/python3.12/dist-packages (from tensorboard~=2.20.0->tensorflow==2.20.0)
Requirement already satisfied: pillow in /usr/local/lib/python3.12/dist-packages (from tensorboard~=2.20.0->tensorflow==2.20.0)
Requirement already satisfied: tensorboard-data-server<0.8.0,>=0.7.0 in /usr/local/lib/python3.12/dist-packages (from tensorboard~=2.20.0->tensorflow==2.20.0)
Requirement already satisfied: werkzeug>=1.0.1 in /usr/local/lib/python3.12/dist-packages (from tensorboard~=2.20.0->tensorflow==2.20.0)
Requirement already satisfied: markupsafe>=2.1.1 in /usr/local/lib/python3.12/dist-packages (from werkzeug>=1.0.1->tensorflow==2.20.0)
```

```
Using cached tensorflow-2.20.0-cp312-cp312-manylinux_2_17_x86_64.manylinux2014_x86_64.whl (620.7 MB)
Using cached tf_keras-2.20.1-py3-none-any.whl (1.7 MB)
Using cached keras-3.15.0-py3-none-any.whl (1.7 MB)
Installing collected packages: keras, tensorflow, tf-keras
Successfully installed keras-3.15.0 tensorflow-2.20.0 tf-keras-2.20.1
```

```
#Shows the tensor flow keras version
import tf_keras
print(tf_keras.__version__)
```

```
2.20.1
```

```
# Import pipeline from transformers library
# A pipeline is used to take our input and process it using a model and return the generated text
from transformers import pipeline
# Path to our HF model
# Roberta is a pretrained for twitter sentiment analysis

model_path = "cardiffnlp/twitter-roberta-base-sentiment-latest"
# Load model into pipeline
# Create a sentiment analysis pipeline using the pretrained model.
pipe = pipeline(
# loads the previously loaded model (Roberta)
model=model_path,
#Load the tokenizer attached to the model
tokenizer=model_path,
# Return the scores for all sentiment classes
# (e.g., Positive, Neutral, and Negative)
return_all_scores=
True
,
# connect device to CPU
device=-1
)
```

```
WARNING:torchao.kernel.intmm:Warning: Detected no triton, on systems without Triton certain kernels will not work
Some weights of the model checkpoint at cardiffnlp/twitter-roberta-base-sentiment-latest were not used when initializing
- This IS expected if you are initializing RobertaForSequenceClassification from the checkpoint of a model trained on a different task
- This IS NOT expected if you are initializing RobertaForSequenceClassification from the checkpoint of a model trained on the same task
/usr/local/lib/python3.12/dist-packages/transformers/pipelines/text_classification.py:106: UserWarning: `return_all_scores` is deprecated and will be removed in a future version. Use `return_token_embeddings` instead.
warnings.warn(
```

```
# Import numpy for numerical calculations
import numpy as np
# Import tqdm library to display the progress bar
from tqdm import tqdm
# Import KeyDataset to efficiently pass only the required column
from transformers.pipelines.pt_utils import KeyDataset
#creates a list to store the predicted labels
y_pred = []
#Run sentiment analysis on the loaded dataset
#KeyDataset extract only text column for analysis
# tqdm display the progress bar
for output in tqdm(pipe(KeyDataset(data["test"], "text")), total=len(data["test"])):

# Get the probability scores for negative and positive
negative_score = output[0]["score"]
positive_score = output[2]["score"]
# Compare the 2 scores and note the index of larger score

assignment = np.argmax([negative_score, positive_score])
# add the predicted label to the created list
y_pred.append(assignment)
```

```
100%|██████████| 1066/1066 [02:57<00:00, 5.99it/s]
```

```
# Import classification_report from sklearn metrics
#It is used to evaluate how well our model performed
from sklearn.metrics import classification_report
# creates a function
def evaluate_performance(y_true, y_pred):
# Create and print the classification report
    performance = classification_report(
        # loads the y actual and y predicted values which are previously stored
        y_true, y_pred,
        #Give meaning full names to the dataset
        # 0 - Negative
        #1 -Positive
        target_names=["Negative Review", "Positive Review"])
    # print the classificatin report
    print(performance)
    # call the function to evaluate the predictions
    evaluate_performance(data["test"]["label"], y_pred)
```

	precision	recall	f1-score	support
Negative Review	0.76	0.88	0.81	533
Positive Review	0.86	0.72	0.78	533
accuracy			0.80	1066
macro avg	0.81	0.80	0.80	1066
weighted avg	0.81	0.80	0.80	1066

```
# Import sentencetransformer from sentence_transformers
# It is used to load the pretrained embedding models that convert text into embeddings
from sentence_transformers import SentenceTransformer
# Load model
# "all-mpnet-base-v2" is a popular model that generates high-quality sentence embeddings.
model = SentenceTransformer("sentence-transformers/all-mpnet-base-v2")
# Convert text to embeddings
train_embeddings = model.encode(data["train"]["text"],
# show_progress_bar=True displays a progress bar while encoding the data.
show_progress_bar=
True
)
# Convert all the test reviews into embeddings
test_embeddings = model.encode(data["test"]["text"],
show_progress_bar=
True
)
```

Batches: 100% 267/267 [15:07<00:00, 1.31it/s]

Batches: 100% 34/34 [01:51<00:00, 1.28s/it]

```
# Shows the shape of train_embeddings
# 8530 Number of training reviews (rows)
# 768- Number of values in each embedding vector (columns)
train_embeddings.shape
```

(8530, 768)

```
# Import Logistic regression from sklearn,linear_model
# Logistic regression is used for classification task
from sklearn.linear_model import LogisticRegression

# Create a Logistic Regression model.
# random_state=42 ensures that the results are reproducible
clf = LogisticRegression(random_state=42)
# Train a logistic regression on our actual train embeddings and labels (0-negative ,1-Positive )
clf.fit(train_embeddings, data["train"]["label"])
```

```
# Predict previously unseen instances
y_pred = clf.predict(test_embeddings)
# Evaluate the predictions by comparing them
evaluate_performance(data["test"]["label"], y_pred)
```

	precision	recall	f1-score	support
Negative Review	0.85	0.86	0.85	533
Positive Review	0.86	0.85	0.85	533
accuracy			0.85	1066
macro avg	0.85	0.85	0.85	1066
weighted avg	0.85	0.85	0.85	1066

```
# Create a text-to-text generation pipeline using the FLAN-T5 model.
# load the text2text model .Which tale input as text and give output in text
pipe = pipeline("text2text-generation",
# Load the pretrained FLAN-T5 Small model from Hugging Face.
# FLAN-T5 is an instruction-tuned model that can perform tasks like summarization, translation, question answer
model="google/flan-t5-small",
# connect to CPU
device=-1
)
```

```
# Prepare our data
prompt = "Is the following sentence positive or negative? "
#map() is a function that performs the same operation on every example (row) in the dataset.
# add a new column called t5 after mapping like "This review is postive or negative + movie review "
data = data.map(
lambda
example: {"t5": prompt + example['text']})
# display the data
data
```

```
DatasetDict({
  train: Dataset({
    features: ['text', 'label', 't5'],
    num_rows: 8530
  })
  validation: Dataset({
    features: ['text', 'label', 't5'],
    num_rows: 1066
  })
  test: Dataset({
    features: ['text', 'label', 't5'],
    num_rows: 1066
  })
})
```

```
# creates a empty list to store the predicted labels
y_pred = []
# KeyDataset extracts the "t5" column, which contains the prompt and the movie review.
# tqdm displays a progress bar while processing.

for output in tqdm(pipe(KeyDataset(data["test"], "t5")), total=len(data["test"])):
#stores the generated text like positive or negative
text = output[0]["generated_text"]
#Convert the text prediction into numerical number
y_pred.append(0
if
text == "negative"
else
1)
```

```
0%|          | 0/1066 [00:00<?, ?it/s]/usr/local/lib/python3.12/dist-packages/transformers/generation/utils.py
warnings.warn(
100%|██████████| 1066/1066 [02:06<00:00, 8.41it/s]
```

```
# Compare the model's predicted labels with the actual (true) labels in the test dataset and evaluate the model
evaluate_performance(data["test"]["label"], y_pred)
```

	precision	recall	f1-score	support
Negative Review	0.83	0.85	0.84	533
Positive Review	0.85	0.83	0.84	533
accuracy			0.84	1066
macro avg	0.84	0.84	0.84	1066
weighted avg	0.84	0.84	0.84	1066

```
# Install or upgrade the Google GenAI
!pip install -U google-genai
```

```
Requirement already satisfied: google-genai in /usr/local/lib/python3.12/dist-packages (2.12.1)
Requirement already satisfied: anyio<5.0.0,>=4.8.0 in /usr/local/lib/python3.12/dist-packages (from google-genai)
Requirement already satisfied: google-auth<2.56.0,>=2.48.1 in /usr/local/lib/python3.12/dist-packages (from google-genai)
Requirement already satisfied: httpx<1.0.0,>=0.28.1 in /usr/local/lib/python3.12/dist-packages (from google-genai)
Requirement already satisfied: pydantic<3.0.0,>=2.12.5 in /usr/local/lib/python3.12/dist-packages (from google-genai)
Requirement already satisfied: requests<3.0.0,>=2.28.1 in /usr/local/lib/python3.12/dist-packages (from google-genai)
Requirement already satisfied: tenacity<9.2.0,>=8.2.3 in /usr/local/lib/python3.12/dist-packages (from google-genai)
Requirement already satisfied: websockets<17.0,>=13.0.0 in /usr/local/lib/python3.12/dist-packages (from google-genai)
Requirement already satisfied: typing-extensions<5.0.0,>=4.14.0 in /usr/local/lib/python3.12/dist-packages (from google-genai)
Requirement already satisfied: distro<2,>=1.7.0 in /usr/local/lib/python3.12/dist-packages (from google-genai) (1.3.1)
Requirement already satisfied: sniffio in /usr/local/lib/python3.12/dist-packages (from google-genai) (1.3.1)
Requirement already satisfied: idna>=2.8 in /usr/local/lib/python3.12/dist-packages (from anyio<5.0.0,>=4.8.0->google-genai)
Requirement already satisfied: pyasn1-modules>=0.2.1 in /usr/local/lib/python3.12/dist-packages (from google-auth<2.56.0,>=2.48.1->google-genai)
Requirement already satisfied: cryptography>=38.0.3 in /usr/local/lib/python3.12/dist-packages (from google-auth<2.56.0,>=2.48.1->google-genai)
Requirement already satisfied: rsa<5,>=3.1.4 in /usr/local/lib/python3.12/dist-packages (from google-auth<2.56.0,>=2.48.1->google-genai)
Requirement already satisfied: certifi in /usr/local/lib/python3.12/dist-packages (from httpx<1.0.0,>=0.28.1->google-genai)
Requirement already satisfied: httpcore==1.* in /usr/local/lib/python3.12/dist-packages (from httpx<1.0.0,>=0.28.1->google-genai)
Requirement already satisfied: h11>=0.16 in /usr/local/lib/python3.12/dist-packages (from httpcore==1.*->httpx<1.0.0,>=0.28.1->google-genai)
Requirement already satisfied: annotated-types>=0.6.0 in /usr/local/lib/python3.12/dist-packages (from pydantic<3.0.0,>=2.12.5->google-genai)
Requirement already satisfied: pydantic-core==2.46.4 in /usr/local/lib/python3.12/dist-packages (from pydantic<3.0.0,>=2.12.5->google-genai)
Requirement already satisfied: typing-inspection>=0.4.2 in /usr/local/lib/python3.12/dist-packages (from pydantic<3.0.0,>=2.12.5->google-genai)
Requirement already satisfied: charset-normalizer<4,>=2 in /usr/local/lib/python3.12/dist-packages (from requests<3.0.0,>=2.28.1->google-genai)
Requirement already satisfied: urllib3<3,>=1.21.1 in /usr/local/lib/python3.12/dist-packages (from requests<3.0.0,>=2.28.1->google-genai)
Requirement already satisfied: cffi>=2.0.0 in /usr/local/lib/python3.12/dist-packages (from cryptography>=38.0.3->google-genai)
Requirement already satisfied: pyasn1<0.7.0,>=0.6.1 in /usr/local/lib/python3.12/dist-packages (from pyasn1-modules>=0.2.1->google-auth<2.56.0,>=2.48.1->google-genai)
Requirement already satisfied: pycparser in /usr/local/lib/python3.12/dist-packages (from cffi>=2.0.0->cryptography>=38.0.3->google-genai)
```

```
#import genAI from google
from google import genai

# Create a Gemini client using your API key
# Replace the API key with your own valid key.
client = genai.Client(api_key="AQ.Ab8RN6IXt9RTm6N1e6IdtrRx_rCNMa8vRG040wSCPubLZ5vW1g")

# Function to generate a response from Gemini
def chatgpt_generation(prompt, document, model="models/gemini-flash-latest"):
    """Generate an output based on a prompt and an input document."""

    # Create a conversation with a system message and a user message
    messages = [
        {
            "role": "system",
            "content": "You are a helpful assistant."
        },
        {
            "role": "user",
            "content": prompt.replace("[DOCUMENT]", document)
        }
    ]

    # Gemini accepts text, so create one text string
```

```

# Create an empty string to store the conversation.
conversation = ""

# Add each message to the conversation
# like role :user ,content : "It is helpful "
# It will display like user :It is helpful
for message in messages:
    conversation += f"{message['role']}: {message['content']}\n"

# Send the conversation to the Gemini model
response = client.models.generate_content(
    model=model,
    contents=conversation
)

# Return only the generated text
return response.text.strip()

# Prompt template
prompt = """
Predict whether the following document is a positive or negative movie review:

[DOCUMENT]

If it is positive return 1 and if it is negative return 0.
Do not give any other answers.
"""

# Input document
document = "unpretentious, charming, quirky, original"

# Call the function and print the prediction using document input
print(chatgpt_generation(prompt, document))

```

1

YEXT CLASSIFICATIONIS DONE IN ABPUE THIS IS WITHOUT USING THE OPENAI like gpt ot gemini

```

# creates a function to predict and load the model
def chatgpt_generation(prompt, document, model="google/flan-t5-small"):
    """Generate an output based on a prompt and an input document."""
# Create a conversation with the system and user
    messages = [
        {
            "role": "system",
            "content": "You are a helpful assistant."
        },
        {
            "role": "user",
            "content": prompt.replace("[DOCUMENT]", document)
        }
    ]
# Extract only the user's prompt from the messages list.
# messages[0] is the system message.
# messages[1] is the user message.
    generation = pipe(
        messages[1]["content"],
        # generate only 50 new tokens
        max_new_tokens=50,
        # Disable random sampling
        do_sample=False
    )
# returns the generated text
    return generation[0]["generated_text"]

```

```
prompt = """Predict whether the following document is a positive
or negative movie review:
```

```
[DOCUMENT]
```

```
If it is positive return 1 and if it is negative return 0.
Do not give any other answers.
"""
```

```
document = "unpretentious, charming, quirky, original"
# call the function and print the prediction
print(chatgpt_generation(prompt, document))
```

```
0
```

```
# You can skip this if you want to save your (free) credits
predictions = [
    # Call the chatgpt_generation() function for each movie review (doc).
    # 1 - Positive review
    # 0 - Negative review
    chatgpt_generation(prompt, doc)
# tqdm display the progress bar
# Loop through every movie review in the "text" column
# of the test dataset
for doc in tqdm(data["test"]["text"])]
```

```
100%|██████████| 1066/1066 [03:18<00:00, 5.36it/s]
```

```
# Extract predictions
y_pred = [
# The model may return:
# "positive" -> Convert to 1
# "negative" -> Convert to 0
# "1" or "0" -> Convert the string to an integer
    1 if pred.lower() == "positive"
    else 0 if pred.lower() == "negative"
    # Convert the strings into integers like "0" to 0
    else int(pred)
    # stores in the predictions list
    for pred in predictions
]
# Compare the predicted labels with the actual labels
# in the test dataset and display the model's performance.
# Evaluate performance
evaluate_performance(data["test"]["label"], y_pred)
```

	precision	recall	f1-score	support
Negative Review	0.51	0.99	0.67	533
Positive Review	0.82	0.03	0.05	533
accuracy			0.51	1066
macro avg	0.66	0.51	0.36	1066
weighted avg	0.66	0.51	0.36	1066

